

JAVA LAB RECORD

Experiment No.	06
Date	23/09/26
Title	Implementation of ArrayList Methods in Java

Objective / Aim

To study and implement commonly used methods of the Java ArrayList class, including adding, accessing, updating, removing, searching, sorting, checking the size and empty state, and clearing elements.

System Requirements

Hardware Requirements

- Processor: Intel Core i3 or equivalent
- RAM: Minimum 4 GB
- Storage: Minimum 10 GB free space
- Keyboard and mouse
- Monitor/display

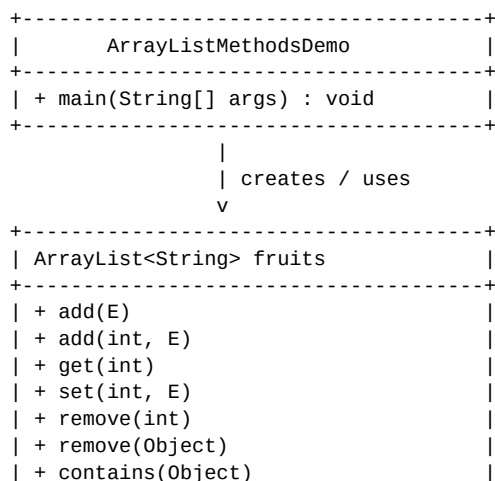
Software Requirements

- Operating System: Windows / Linux / macOS
- Java Development Kit (JDK): 17 or later
- IDE: Eclipse / IntelliJ IDEA / NetBeans / VS Code
- Command Prompt / Terminal

Algorithm / Logic Description

1. Start the program.
2. Import ArrayList and Comparator from java.util.
3. Create an ArrayList named fruits to store String values.
4. Add four fruit names using add(E e).
5. Insert a fruit at a particular position using add(index, element).
6. Retrieve an element using get(index).
7. Replace an element using set(index, element).
8. Remove an element by index using remove(index).
9. Remove an element by value using remove(object).
10. Check whether an element exists using contains(object).
11. Find the number of elements using size() and check whether the list is empty using isEmpty().
12. Add another duplicate value and find its first and last positions using indexOf() and lastIndexOf().
13. Sort the list in natural order using sort(Comparator.naturalOrder()).
14. Remove all elements using clear() and verify the list is empty.
15. Stop the program.

UML Class Diagram



```
| + size() |
| + isEmpty() |
| + indexOf(Object) |
| + lastIndexOf(Object) |
| + sort(Comparator) |
| + clear() |
+-----+
```

Source Code

```
import java.util.ArrayList;
import java.util.Comparator;

class ArrayListMethodsDemo {

    public static void main(String[] args) {

        // Creating an ArrayList
        ArrayList<String> fruits = new ArrayList<>();

        // add(E e)
        fruits.add("Apple");
        fruits.add("Mango");
        fruits.add("Banana");
        fruits.add("Orange");

        System.out.println("After add(): " + fruits);

        // add(int index, E element)
        fruits.add(2, "Grapes");
        System.out.println("After add(index, element): " + fruits);

        // get(int index)
        System.out.println("Element at index 1: " + fruits.get(1));

        // set(int index, E element)
        fruits.set(1, "Pineapple");
        System.out.println("After set(): " + fruits);

        // remove(int index)
        fruits.remove(3);
        System.out.println("After remove(index): " + fruits);

        // remove(Object o)
        fruits.remove("Apple");
        System.out.println("After remove(object): " + fruits);

        // contains(Object o)
        System.out.println("Contains Banana? " + fruits.contains("Banana"));

        // size()
        System.out.println("Size of ArrayList: " + fruits.size());

        // isEmpty()
        System.out.println("Is ArrayList empty? " + fruits.isEmpty());

        // indexOf(Object o)
        fruits.add("Banana");
        System.out.println("First index of Banana: " + fruits.indexOf("Banana"));

        // lastIndexOf(Object o)
        System.out.println("Last index of Banana: " + fruits.lastIndexOf("Banana"));

        // sort(Comparator)
        fruits.sort(Comparator.naturalOrder());
        System.out.println("After sorting: " + fruits);

        // clear()
        fruits.clear();
        System.out.println("After clear(): " + fruits);

        // Checking isEmpty() again
        System.out.println("Is ArrayList empty now? " + fruits.isEmpty());
    }
}
```

```
}
```

Completion and Execution Commands

1. Save the program as: ArrayListMethodsDemo.java
2. Open Command Prompt / Terminal and move to the folder containing the file.
3. Compile:
`javac ArrayListMethodsDemo.java`
4. Execute:
`java ArrayListMethodsDemo`
5. The results of the ArrayList operations are displayed on the console.

Output

```
After add(): [Apple, Mango, Banana, Orange]
After add(index, element): [Apple, Mango, Grapes, Banana, Orange]
Element at index 1: Mango
After set(): [Apple, Pineapple, Grapes, Banana, Orange]
After remove(index): [Apple, Pineapple, Grapes, Orange]
After remove(object): [Pineapple, Grapes, Orange]
Contains Banana? false
Size of ArrayList: 3
Is ArrayList empty? false
First index of Banana: 3
Last index of Banana: 3
After sorting: [Banana, Grapes, Pineapple]
After clear(): []
Is ArrayList empty now? true
```

Result

Thus, the Java program was successfully implemented and executed to demonstrate the commonly used methods of the ArrayList class.